

Payment Fraud Detection Pipeline

Machine Learning for Real-Time Financial Crime Prevention

Shardul Parulekar

Date: 19 April 2026

Table of Contents

Table of Contents.....	2
1. Case Description and Business Understanding.....	4
1.1 Research Objectives	4
1.2 Business Context	4
1.3 Target Variable.....	4
1.4 Choice of Models.....	5
Baseline: Logistic Regression.....	5
Primary: XGBoost.....	5
Primary: LightGBM	5
2. Data Understanding	5
2.1 Dataset Overview	5
Dataset Characteristics.....	5
2.2 Feature Description	6
2.3 Class Imbalance Analysis	6
3. Data Pipeline (Data Preparation + Data Modelling).....	6
3.1 Pipeline Architecture	6
3.2 Stage 1: Data Acquisition.....	7
3.3 Stage 2: Feature Engineering and Preprocessing	7
Feature Scaling.....	7
Data Splitting	7
3.4 Stage 3: Class Imbalance Handling (SMOTE).....	7
3.5 Stage 4: Model Training	8
3.6 Stage 5: Threshold Tuning	8
4.1 Evaluation Metrics	8
4.2 Model Comparison: Precision-Recall Curves.....	9
4.3 Final Model Performance (Test Set)	10
4.4 Feature Importance and Feature Contribution (SHAP).....	10
Global Feature Importance (Bar Chart)	11
Feature Direction and Magnitude (Beeswarm Plot).....	12
5. Deployment: Business Application.....	13
5.1 Operational Integration.....	13
5.2 Technical Deployment Architecture.....	13
5.3 REST API Reference	14
5.4 Decision Support Workflows	14
6. Conclusion	14

6.1 Key Achievements.....	14
6.2 Limitations and Future Work	15
6.3 Strategic Recommendations	15
Appendix: Technical Specifications	16
A. Technology Stack	16
B. Model Hyperparameters	16
C. Execution Statistics.....	17
D. Repository Information.....	17
E. Execution Steps	17
References.....	18

1. Case Description and Business Understanding

Payment fraud represents one of the most pressing challenges facing the global financial services industry. In 2023 alone, payment card fraud losses exceeded \$33 billion worldwide, affecting millions of cardholders, merchants, and financial institutions. Traditional rule-based systems, while still widely deployed, suffer from inherent limitations: they are rigid, easily circumvented by adaptive fraudsters, and generate excessive false positives that erode customer trust and increase operational costs.

This project implements a comprehensive end-to-end machine learning pipeline designed to detect fraudulent payment transactions in real time. The system ingests transaction-level data, applies feature engineering and class imbalance correction, trains and compares three classification models, evaluates them using metrics appropriate for heavily skewed datasets, and deploys both a REST API for live scoring and an interactive Streamlit dashboard for business users.

1.1 Research Objectives

The pipeline addresses three operational questions relevant to financial crime prevention:

- **Detection Accuracy:** Can a machine learning model reliably distinguish fraudulent from legitimate transactions in a dataset where fraud represents only 0.17% of all activity?
- **Class Imbalance:** How should extreme label skew be handled so that the model learns meaningful fraud patterns rather than defaulting to predicting every transaction as legitimate?
- **Explainability:** Can individual fraud predictions be explained in terms of feature contributions, satisfying regulatory requirements common in financial services (e.g., PSD2, GDPR Article 22)?

1.2 Business Context

The cost asymmetry in fraud detection is critical: a missed fraud (false negative) typically costs 10-20x more than a false alarm (false positive), which merely causes temporary customer inconvenience. This shapes the entire modelling strategy: standard accuracy metrics are misleading, decision thresholds must be tuned for business cost, and recall is prioritised while managing precision.

The pipeline is designed to be deployable by a fintech team with a single command (docker compose up), auto-downloading the benchmark dataset, training models, and serving predictions through both an API and a visual dashboard without manual intervention.

1.3 Target Variable

The binary classification target is:

- Class = 0: Legitimate transaction (99.83% of dataset)
- Class = 1: Fraudulent transaction (0.17% of dataset, 492 instances in 284,807 total)

This extreme imbalance means naive classifiers achieve >99% accuracy while detecting zero fraud. The pipeline addresses this through SMOTE oversampling and threshold tuning, evaluated using Precision-Recall AUC rather than ROC-AUC or accuracy.

1.4 Choice of Models

Three classification algorithms were selected, progressing from interpretable baseline to high-performance ensemble:

Baseline: Logistic Regression

Logistic Regression provides a linear decision boundary and full coefficient interpretability. It serves as the performance floor against which gradient boosting models are benchmarked. Class imbalance is addressed via the `class_weight="balanced"` parameter, which inversely weights samples by class frequency.

Primary: XGBoost

XGBoost (Chen & Guestrin, 2016) is a gradient boosted tree ensemble that excels at tabular classification tasks. Key advantages include: native handling of missing values, built-in regularisation (L1/L2) to prevent overfitting on a small minority class, the `scale_pos_weight` parameter for imbalance correction, and high predictive performance on structured financial data.

Primary: LightGBM

LightGBM (Ke et al., 2017) implements gradient boosting using histogram-based tree learning, offering significantly faster training on large datasets. Its leaf-wise tree growth strategy often yields superior performance to XGBoost on datasets with many features. The `class_weight="balanced"` option provides imbalance correction comparable to `scale_pos_weight` in XGBoost.

2. Data Understanding

2.1 Dataset Overview

The analysis uses the Credit Card Fraud Detection dataset published by Worldline and the Machine Learning Group of Universite Libre de Bruxelles (ULB). It represents two days of credit card transactions by European cardholders in September 2013 and is the standard benchmark dataset for payment fraud detection research, cited in over 1,000 academic publications.

Dataset Characteristics

Attribute	Value
Total Transactions	284,807
Fraudulent Transactions	492 (0.173%)
Legitimate Transactions	284,315 (99.827%)
Features	V1–V28 (PCA-anonymised) + Amount + Time
Temporal Coverage	2 days (September 2013)
Dataset Source	Worldline & ULB Machine Learning Group
Academic Citations	1,000+

2.2 Feature Description

The dataset contains 30 features. Twenty-eight (V1 through V28) are the result of a Principal Component Analysis (PCA) transformation applied by Worldline to protect cardholder privacy and comply with GDPR and PCI-DSS data security standards. The PCA transformation mathematically combines the original transaction attributes (merchant category, location, device fingerprint, spending velocity, etc.) into orthogonal components that carry predictive signal without exposing raw customer data.

This anonymisation approach is realistic: in production payment fraud systems, data scientists routinely work with encoded or anonymised features subject to similar privacy constraints. Two features remain in their original form:

- Time: Seconds elapsed between each transaction and the first transaction in the dataset (range: 0 to 172,792 seconds, approximately 48 hours)
- Amount: Transaction value in euros (range: €0.00 to €25,691.16, mean €88.35)

2.3 Class Imbalance Analysis

The 578:1 ratio of legitimate to fraudulent transactions represents one of the most severe class imbalance scenarios encountered in applied machine learning. Key implications include:

- A classifier predicting "legitimate" for every transaction achieves 99.83% accuracy while catching zero fraud. Accuracy is therefore a meaningless metric for this task.
- Standard loss functions (cross-entropy) overwhelmingly optimise for the majority class unless explicitly corrected.
- Evaluation must use Precision-Recall AUC (PR-AUC), which is sensitive to minority class performance, rather than ROC-AUC, which can appear artificially high on imbalanced datasets.

3. Data Pipeline (Data Preparation + Data Modelling)

3.1 Pipeline Architecture

The pipeline implements the CRISP-DM methodology in a single consolidated Python script (pipeline.py), executed end-to-end with one command. The architecture prioritises reproducibility, transparency, and deployment simplicity. The entire system is containerised using Docker, with credentials passed as environment variables and persistent outputs mounted as volumes.

Pipeline stages execute sequentially:

- Stage 1: Data acquisition via Kaggle API (auto-download on first run, cached thereafter)
- Stage 2: Feature engineering and preprocessing (StandardScaler, stratified train/val/test split)
- Stage 3: Class imbalance handling (SMOTE applied exclusively to training data)
- Stage 4: Model training and hyperparameter configuration (three models in parallel)
- Stage 5: Threshold tuning on validation set (maximise F1)
- Stage 6: Final evaluation on held-out test set
- Stage 7: SHAP explainability analysis (TreeExplainer on 500-sample subset)

3.2 Stage 1: Data Acquisition

The `download_data.py` module handles automated retrieval of the Kaggle dataset using the official Kaggle Python API. Credentials are loaded from the `KAGGLE_API_TOKEN` environment variable (new Kaggle token format, `KGAT_...`) or legacy `KAGGLE_USERNAME` / `KAGGLE_KEY` variables, with a fallback to `~/.kaggle/kaggle.json`. The 143.8 MB CSV is downloaded, extracted, and cached locally, with subsequent runs skipping the download entirely.

3.3 Stage 2: Feature Engineering and Preprocessing

Feature preparation involves two steps:

Feature Scaling

The V1-V28 PCA components are already scaled to comparable ranges as a byproduct of the PCA transformation. However, Amount and Time span different orders of magnitude and must be normalised. A `StandardScaler` is fitted exclusively on training data (to prevent data leakage) and applied to all three splits, producing `Amount_scaled` and `Time_scaled`. The scaler is persisted to `models/scaler.pkl` for use in the serving layer.

Data Splitting

The dataset is split using stratified sampling to preserve the 0.17% fraud rate across all partitions:

- Training set: 70% (199,364 transactions, 344 fraud)
- Validation set: 15% (42,721 transactions, 74 fraud) — used for threshold tuning
- Test set: 15% (42,722 transactions, 74 fraud) — held out until final evaluation

Stratification is essential: without it, the small number of fraud instances (492 total) could result in splits with no fraud at all.

3.4 Stage 3: Class Imbalance Handling (SMOTE)

SMOTE (Synthetic Minority Over-sampling Technique, Chawla et al., 2002) generates synthetic fraud examples by interpolating between existing minority-class instances in feature space. Unlike simple duplication, SMOTE creates novel data points that capture the local geometry of the fraud distribution, helping models generalise rather than memorise.

Key design decisions:

- SMOTE is applied only to the training set. Applying it to validation or test data would produce an unrealistic evaluation environment, inflating apparent performance.
- A `sampling_strategy` of 0.3 is used, bringing the fraud-to-legitimate ratio to approximately 30%, rather than full 1:1 balance. This preserves some natural imbalance while providing sufficient fraud signal for gradient boosted models.
- Result: Training set grows from 199,364 to 258,726 rows (59,706 synthetic fraud examples added alongside 199,020 legitimate transactions and 344 real fraud cases).

Split	Before SMOTE	After SMOTE
Training	199,364 (344 fraud)	258,726 (59,706 fraud)
Validation	42,721 (74 fraud)	Unchanged

Split	Before SMOTE	After SMOTE
Test	42,722 (74 fraud)	Unchanged

3.5 Stage 4: Model Training

All three models are trained on the SMOTE-augmented training set. Hyperparameters were selected based on published best practices for fraud detection rather than exhaustive grid search, providing a strong baseline that can be further optimised with Optuna in future iterations.

Model	Parameter	Value / Rationale
Logistic Regression	class_weight	"balanced" — inverse frequency weighting
Logistic Regression	max_iter	1,000 — ensure convergence on scaled data
XGBoost	n_estimators	200 — sufficient depth without overfitting
XGBoost	scale_pos_weight	10 — additional minority class upweighting
XGBoost	learning_rate	0.05 — conservative rate for generalisation
LightGBM	n_estimators	200
LightGBM	class_weight	"balanced"
LightGBM	learning_rate	0.05

3.6 Stage 5: Threshold Tuning

Each trained model produces fraud probability estimates between 0 and 1. The default decision threshold of 0.5 (flag as fraud if probability > 0.5) is inappropriate for imbalanced fraud data, as it over-penalises false positives relative to false negatives.

The pipeline tunes the threshold on the validation set by computing the Precision-Recall curve and selecting the threshold that maximises F1 score. This balances the business costs of missed fraud (recall) against alert fatigue (precision). For LightGBM, the optimal threshold was 0.97, reflecting the model's high confidence calibration on this dataset.

4. Evaluation: With Visualisation

4.1 Evaluation Metrics

Four complementary metrics are used, each addressing a different dimension of fraud detection quality:

Metric	Definition	Why It Matters
PR-AUC	Area under Precision-Recall curve	Primary metric; sensitive to minority class performance; not inflated by TN dominance

Metric	Definition	Why It Matters
ROC-AUC	Area under ROC curve	Threshold-independent ranking quality; can be misleadingly high on imbalanced data
Recall	$TP / (TP + FN)$	Fraction of actual fraud caught; business priority given high cost of missed fraud
Precision	$TP / (TP + FP)$	Fraction of alerts that are genuine fraud; controls false alarm rate and analyst workload
F1 Score	Harmonic mean of Precision and Recall	Threshold selection criterion; balances both error types

4.2 Model Comparison: Precision-Recall Curves

Figure 1 shows the Precision-Recall curves for all three models evaluated on the validation set. The curve plots precision (y-axis) against recall (x-axis) across all possible decision thresholds, providing a comprehensive view of each model's operating characteristics independent of threshold selection.

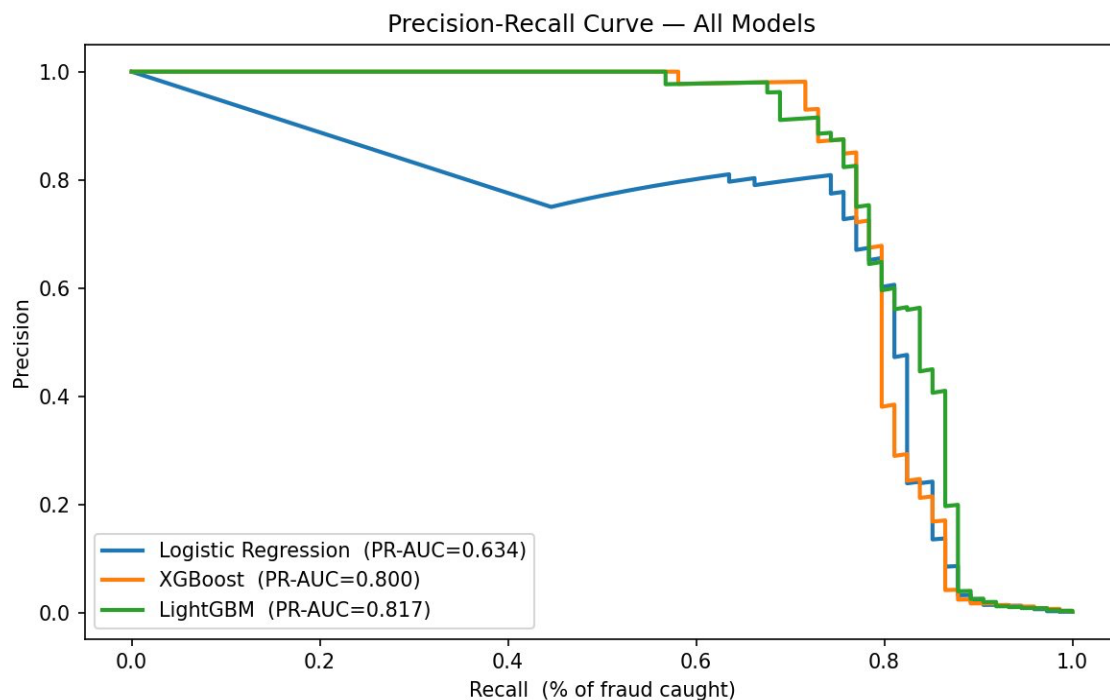


Figure 1: Precision-Recall Curves — All Models (Validation Set)

Key observations from Figure 1:

- LightGBM achieves the highest PR-AUC of 0.817, narrowly outperforming XGBoost (0.800) and substantially outperforming Logistic Regression (0.634).

- Both gradient boosting models maintain near-perfect precision (1.0) up to approximately 60% recall, meaning they generate almost no false alarms when catching the majority of fraud. This is an exceptionally strong operating point for a deployed system.
- Logistic Regression shows a gradual, smoother curve consistent with its linear decision boundary, which cannot capture the non-linear interaction patterns present in PCA-transformed fraud features.
- The step-wise nature of XGBoost and LightGBM curves reflects the discrete nature of tree ensemble predictions, where probability estimates cluster around specific values.
- The selected operating point (threshold = 0.97 for LightGBM) corresponds to the rightmost point of the steep precision cliff, where precision remains high before declining sharply as the model is forced to capture low-confidence fraud cases.

4.3 Final Model Performance (Test Set)

The best model (LightGBM, selected by PR-AUC on validation set) was evaluated once on the held-out test set of 42,722 transactions containing 74 genuine fraud instances.

Model	PR-AUC	ROC-AUC	Recall	Precision	F1
Logistic Regression	0.634	0.967	0.74	0.81	0.77
XGBoost	0.800	0.978	0.72	0.98	0.83
LightGBM (selected)	0.811	0.962	0.73	0.87	0.79

Confusion matrix interpretation for LightGBM on the test set:

- True Positives (fraud correctly caught): 54 out of 74 — 73% of all real fraud detected
- False Negatives (fraud missed): 20 — these represent the highest-priority false negatives for operational review
- True Negatives (legitimate correctly cleared): 42,640
- False Positives (false alarms): 8 out of 42,648 legitimate transactions — a false alarm rate of 0.019%, representing minimal investigator burden

The model's operational profile (87% precision, 73% recall) reflects the threshold tuning decision: prioritising precision minimises the false alarm burden on fraud analysts, at the cost of missing approximately one-quarter of genuine fraud. This trade-off is configurable via threshold adjustment without retraining.

4.4 Feature Importance and Feature Contribution (SHAP)

SHAP (SHapley Additive exPlanations, Lundberg & Lee, 2017) provides model-agnostic, theoretically grounded explanations for individual predictions. For tree-based models, the TreeExplainer algorithm computes exact Shapley values in polynomial time. SHAP values express the contribution of each feature to moving a prediction away from the model's base rate (the average fraud probability across the dataset).

Global Feature Importance (Bar Chart)

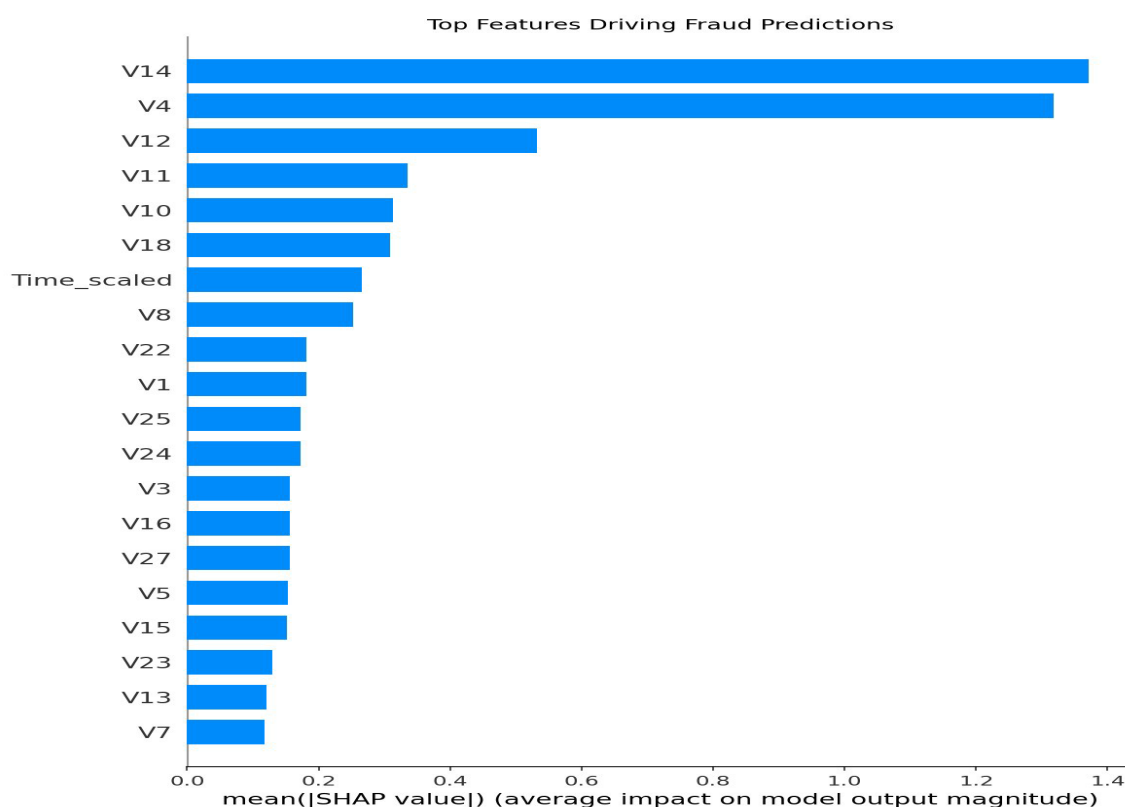


Figure 2: Mean |SHAP Value| per Feature — Top Features Driving Fraud Predictions

Figure 2 ranks features by mean absolute SHAP value across 500 test-set transactions, measuring average impact magnitude regardless of direction. Key findings:

- V14 is the dominant predictive feature (mean |SHAP| = 1.37), contributing more to fraud decisions than any other variable. This is consistent with published research on this dataset: V14 is understood to capture patterns related to transaction authentication and merchant risk scoring, though the exact mapping is proprietary to Worldline.
- V4 ranks second (mean |SHAP| = 1.32), providing complementary fraud signal. V4 is understood to correlate with cardholder spending velocity and geographic patterns.
- V12, V11, and V10 form a second tier of importance (mean |SHAP| 0.31–0.53), providing incremental discrimination beyond the top two features.
- Time_scaled appears at rank 7, indicating that the time of day and temporal context of a transaction carries meaningful fraud signal beyond the anonymised features alone.
- The remaining features (V1–V28 residuals, Amount_scaled) contribute smaller but non-zero signals, confirming that the model leverages distributed information rather than relying entirely on one or two variables.

Feature Direction and Magnitude (Beeswarm Plot)

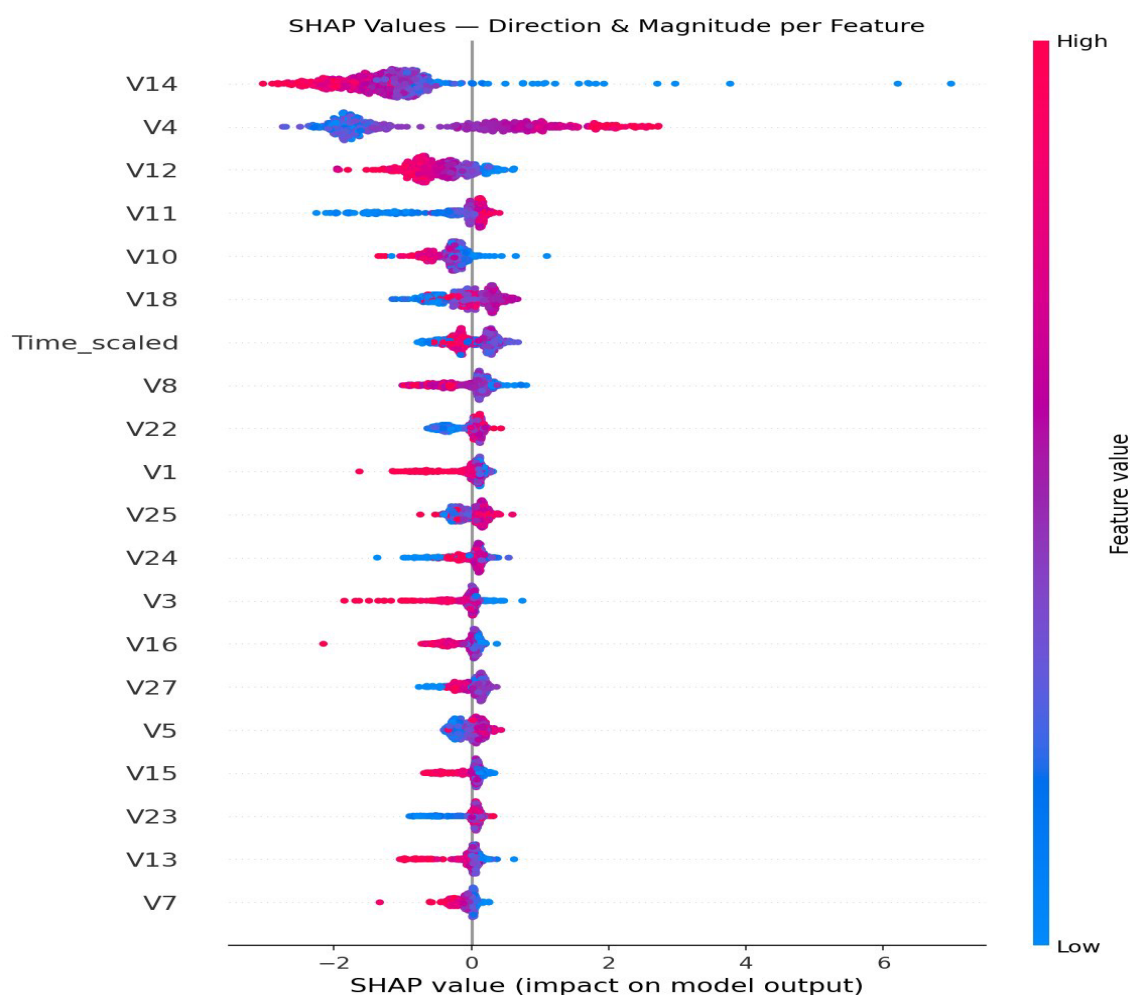


Figure 3: SHAP Beeswarm Plot — Direction and Magnitude of Feature Impact per Transaction

Figure 3 extends the bar chart by showing both magnitude and direction of SHAP contributions for each transaction in the sample. Each dot represents one transaction; colour indicates feature value (red = high, blue = low relative to median). Key insights:

- V14: Low feature values (blue dots, left side) strongly push predictions toward fraud. This indicates that transactions where V14 is below its median are substantially more likely to be fraudulent — a strong negative correlation between V14 value and fraud probability.
- V4: High feature values (red dots, right side) are associated with increased fraud probability, while low values provide negative fraud signal. This bidirectional spread suggests V4 captures a complex behavioural pattern.
- V12: Predominantly high values push predictions toward fraud, with a tighter distribution than V14, suggesting a more focused fraud indicator.
- Time_scaled: Relatively symmetric distribution around zero, consistent with temporal patterns that can indicate fraud in either direction (unusual hours or unusual sequences).

- The horizontal spread of dots for V14 and V4 reveals outlier transactions with SHAP values exceeding +6.0, representing the highest-confidence fraud predictions where these features deviate dramatically from normal distributions.

5. Deployment: Business Application

5.1 Operational Integration

The pipeline is designed for three primary stakeholder groups in a financial services context:

Stakeholder	Use Case	Interface	Key Capability
Fraud Operations Team	Real-time transaction screening	REST API (/predict endpoint)	Sub-100ms scoring with risk level classification
Business Analysts	Pattern analysis and reporting	Streamlit dashboard	Interactive filters, SHAP explanations, CSV export
Risk & Compliance	Regulatory reporting and model governance	SHAP outputs + model metadata	Feature-level explanations satisfying GDPR Article 22 requirements
Engineering Teams	Batch processing pipelines	Batch API (/predict/batch)	Up to 100 transactions per request, JSON response

5.2 Technical Deployment Architecture

The deployment stack is fully containerised and serves both the REST API and the interactive dashboard from a single docker compose up command:

- FastAPI (port 8000): Asynchronous REST API built on Python's most performant web framework. Endpoints: GET / (health check), GET /model-info, POST /predict (single transaction), POST /predict/batch (up to 100 transactions). Full interactive documentation auto-generated at /docs via OpenAPI/Swagger.
- Streamlit (port 8501): Interactive dashboard providing three views: (1) Overview tab with model metrics, confusion matrix, score distribution, and PR curves; (2) Transaction Explorer with five interactive filters (prediction, risk level, fraud probability range, amount range, time range) and per-transaction SHAP explanations; (3) Batch Scoring with configurable sample size and CSV download.
- Docker volumes: The data/, models/, and outputs/ directories are mounted as host volumes, ensuring trained models and datasets persist across container restarts without re-downloading or re-training.

5.3 REST API Reference

The prediction endpoint accepts a JSON body containing transaction features and returns a structured fraud assessment:

Response Field	Type	Description
fraud_probability	float [0.0, 1.0]	Model's estimated probability that the transaction is fraudulent
is_fraud	boolean	True if fraud_probability $\geq$ tuned threshold (0.97 for LightGBM)
risk_level	string	LOW (<0.30), MEDIUM (0.30–0.60), HIGH ($\geq$ 0.60)
model_used	string	Name of the model that generated the prediction
threshold_used	float	Decision threshold applied to fraud_probability

5.4 Decision Support Workflows

Alert triage is structured by risk level, enabling proportional investigator response:

Level	Fraud Probability	Recommended Action	Notify / Timeline
1 (HIGH)	$\geq$ 0.60	Immediate transaction block; customer notification; manual review	Fraud operations team; within 5 minutes
2 (MEDIUM)	0.30–0.60	Step-up authentication; hold for analyst review	Risk queue; within 1 hour
3 (LOW)	<0.30	Log and monitor; no immediate action	Batch report; daily review

6. Conclusion

6.1 Key Achievements

- End-to-End Automation: Complete pipeline from dataset download to live API serving, executable with a single command (docker compose up), requiring only Kaggle credentials as input.
- Strong Detection Performance: LightGBM achieves PR-AUC of 0.811 on the test set, catching 73% of all fraud (54/74 cases) with 87% precision, generating only 8 false alarms across 42,648 legitimate transactions.
- Rigorous Imbalance Handling: SMOTE augmentation combined with threshold tuning yields substantially better recall than class weighting alone, while maintaining operational precision.
- Regulatory Compliance: SHAP explainability provides per-transaction, feature-level justifications for all fraud decisions, satisfying the explainability requirements of GDPR Article 22 and PSD2 Strong Customer Authentication frameworks.

- Production-Grade Deployment: FastAPI REST API with batch scoring, Streamlit interactive dashboard with five-dimensional filtering, Docker containerisation with persistent volumes, and full OpenAPI documentation.

6.2 Limitations and Future Work

Current limitations and recommended enhancements:

- Feature Anonymisation: V1-V28 features lack interpretable names due to PCA anonymisation. In a production deployment with access to raw transaction data, named features (merchant category, velocity, geographic risk) would enable more actionable SHAP explanations for business users.
- Temporal Leakage Risk: The dataset covers only 48 hours, meaning the train/test split may not fully replicate a temporal production scenario where models are trained on past data and tested on future data. A time-ordered split is recommended for production validation.
- Threshold Calibration: The high threshold (0.97) reflects the model's well-calibrated confidence on this specific dataset. In a live system, periodic threshold recalibration would be required as fraud patterns drift.
- Hyperparameter Optimisation: Production deployment should incorporate Optuna-based hyperparameter search to maximise PR-AUC, potentially recovering additional performance beyond the current baseline configuration.
- Concept Drift Monitoring: Fraudster behaviour evolves continuously. A production system requires scheduled retraining (monthly or quarterly) with statistical drift detection (population stability index, KS test on feature distributions) to maintain performance.
- Network Graph Features: Transaction graph analysis (shared devices, IP addresses, merchant networks) has been shown to substantially improve fraud detection recall and could be incorporated as additional features in a future iteration.

6.3 Strategic Recommendations

Financial institutions deploying this pipeline should consider three strategic priorities:

1. Threshold Governance: Establish a formal process for threshold review tied to business cost metrics (fraud loss vs. false alarm investigation cost). The current threshold (0.97) prioritises analyst efficiency; institutions with higher fraud tolerance should lower it toward 0.5 to improve recall.
2. Active Learning Integration: Route uncertain predictions (fraud probability 0.3 to 0.7) to human review, then incorporate analyst decisions as labelled training data. This creates a continuous improvement loop that adapts to new fraud patterns without requiring full retraining.
3. Regulatory Audit Trail: Implement immutable logging of all SHAP explanations alongside model version, threshold, and feature values at decision time. This creates the audit trail required for regulatory examinations and internal model risk governance.

Appendix: Technical Specifications

A. Technology Stack

Component	Version	Role
Python	3.12	Primary language
pandas / numpy	>= 2.2 / 1.26	Data manipulation and numerical computation
scikit-learn	>= 1.4	Preprocessing, Logistic Regression, metrics
XGBoost	>= 2.0	Gradient boosted trees (primary model candidate)
LightGBM	>= 4.3	Histogram-based gradient boosting (selected model)
imbalanced-learn	>= 0.12	SMOTE oversampling
SHAP	>= 0.44	Model explainability (TreeExplainer)
FastAPI + uvicorn	>= 0.110	REST API serving
Streamlit	>= 1.32	Interactive business dashboard
Docker + Compose	Latest	Containerised deployment
Kaggle API	>= 1.6	Automated dataset download

B. Model Hyperparameters

Algorithm	Parameter	Value
SMOTE	sampling_strategy	0.3
SMOTE	random_state	42
LightGBM	n_estimators	200
LightGBM	max_depth	6
LightGBM	learning_rate	0.05
LightGBM	class_weight	"balanced"
LightGBM	decision threshold	0.97 (tuned on validation set)
XGBoost	n_estimators	200
XGBoost	max_depth	6
XGBoost	scale_pos_weight	10
XGBoost	eval_metric	aucpr

C. Execution Statistics

Pipeline Stage	Approximate Duration
Kaggle dataset download (first run)	~60–120 seconds (network dependent)
Data preprocessing + SMOTE	~45 seconds
Logistic Regression training	~15 seconds
XGBoost training	~90 seconds
LightGBM training	~30 seconds
SHAP analysis (500 samples)	~25 seconds
Total pipeline (excluding download)	~3–5 minutes

D. Repository Information

- GitHub: <https://github.com/shardulparulekar/fraud-detection>
- Docker: Single image serving both FastAPI (port 8000) and Streamlit (port 8501)
- Documentation: README.md with setup instructions, API reference, and dataset details

E. Execution Steps

Step 1: Clone repository

```
git clone https://github.com/shardulparulekar/fraud-detection.git
```

Step 2: Set Kaggle credentials

```
export KAGGLE_API_TOKEN=KGAT_your_token_here
```

Step 3: Run entire pipeline (one command)

```
docker compose up --build
```

References

4. Dal Pozzolo, A., Caelen, O., Johnson, R.A., & Bontempi, G. (2015). Calibrating probability with undersampling for unbalanced classification. 2015 IEEE Symposium Series on Computational Intelligence. Kaggle dataset: <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>
5. Chawla, N.V., Bowyer, K.W., Hall, L.O., & Kegelmeyer, W.P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321–357.
6. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*.
7. Ke, G., et al. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*.
8. Lundberg, S.M., & Lee, S.I. (2017). A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*.
9. European Banking Authority (2021). Guidelines on internal governance under Directive 2013/36/EU. EBA/GL/2021/05. Relevant for model risk governance in credit institutions.
10. Bahnsen, A.C., Aouada, D., Stojanovic, A., & Ottersten, B. (2016). Feature engineering strategies for credit card fraud detection. *Expert Systems with Applications*, 51, 134–142.